INVERSIVE LABS

Prepared for

Juan Manuel Sánchez
Jorge Rodriguez
Mixoor.fun

Prepared by

Sergio Corrales
Ignacio Sanmillan
Inversive Labs

February 9th 2026

ILAR-2026-03

Mixoor Protocol - Solana

Smart Contract Security Assessment

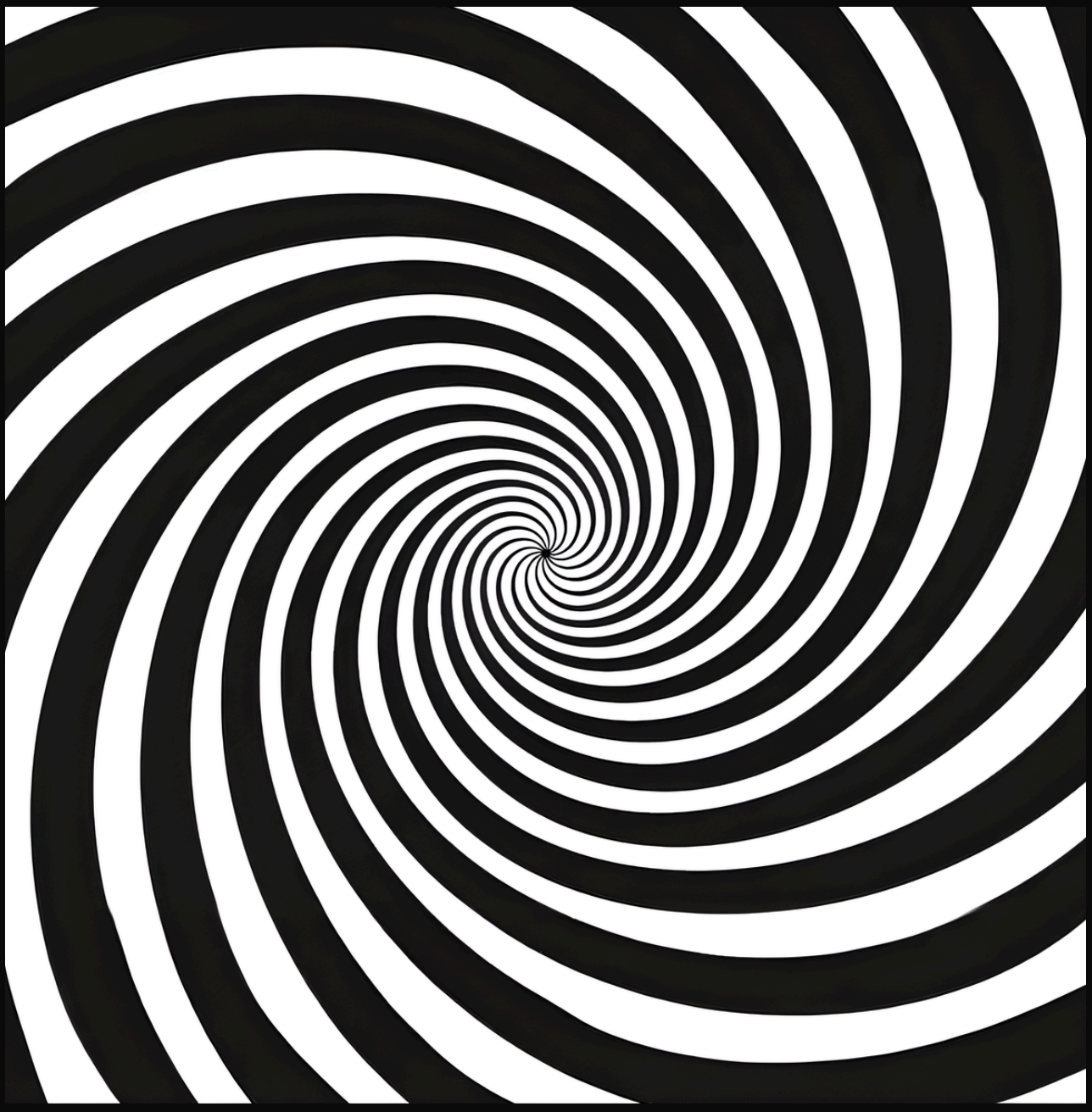


Table of Contents

1. About Inversive
2. Disclaimer
3. Risk Classification
4. Protocol Summary
5. Scope
6. Executive Summary
7. Audit Revision Timeline
8. Summary of Findings
9. Detailed Findings
    - ILRV-2026-03.01: Mint-Agnostic Deposits and Withdrawals Enable Direct Theft of Pool Funds
    - ILRV-2026-03.02: Trusted Setup Ceremony Contains Insufficient Participants
    - ILRV-2026-03.03: Arbitrary Deposits Can Overwrite Merkle Roots and Freeze All Withdrawals
    - ILRV-2026-03.04: Missing Relayer Fee Transfer for SPL Token Withdrawals
    - ILRV-2026-03.05: Admin Withdraw Lacks Validation Enabling Fund Theft and Pool Uninitialization
    - ILRV-2026-03.06: Unrestricted Pool Initialization Enables PDA Squatting and Future Token Griefing
    - ILRV-2026-03.07: Missing Bit Length Constraints Enable Field Overflow in Amount Validation
    - ILRV-2026-03.08: Admin Withdraw Does Not Update Pool Accounting
    - ILRV-2026-03.09: Missing Merkle Tree Capacity Check Causes Permanent Fund Loss After 1M Deposits
    - ILRV-2026-03.10: Missing Authorization Check Allows Malicious Relayers
    - ILRV-2026-03.11: Root History Window Shrinks from 50 to 30 Entries
    - ILRV-2026-03.12: Security Comment May Benefit from Clarification

1. About Inversive Labs

Inversive Labs is a boutique security research firm dedicated to securing emerging blockchain ecosystems. We conduct security assessments for smart contracts and blockchain infrastructure, while also providing custom security tooling and tailored development services.

Prior to Inversive Labs, our founders established themselves as independent security researchers, discovering critical vulnerabilities in production protocols and contributing to the security of major DeFi and open-source projects. Our engineers bring diverse backgrounds in offensive security, low-level exploitation, and smart contract security.

Our expertise spans traditional security and blockchain attack vectors, enabling us to identify critical vulnerabilities across complex protocol implementations and contribute meaningful security research to rapidly evolving ecosystems.

For more on Inversive Labs' security research and services, visit https://inversive.xyz, reach us on X at @inversive_xyz or send us an email at contact@inversive.xyz

2. Disclaimer

This document presents the findings from a security assessment conducted on Mixoor Solana Protocol over a 4-week period.

The analysis represents the state of the codebase at a specific point in time and does not account for subsequent modifications.

This report is provided for informational purposes only and does not constitute investment advice, legal counsel, or a guarantee of security. Inversive Labs makes no warranties, express or implied, regarding the absence of vulnerabilities beyond those identified during the testing period. The deployment and use of the analyzed code remain at the sole risk and responsibility of the implementing parties.

3. Risk Classification

Our audit uses the following severity classification system:

SEVERITY	DESCRIPTION
CRITICAL	Issues that can lead to loss of funds, protocol insolvency, or complete system compromise. Immediate action required.
HIGH	Issues that can lead to significant loss of funds or major protocol disruption under certain conditions. Should be fixed promptly.
MEDIUM	Issues that can lead to unexpected behavior, minor loss of funds, or degraded user experience. Should be addressed.
LOW	Issues that represent potential risks or deviations from best practices. Recommended to fix.
INFORMATIONAL	Code quality improvements, gas optimizations, or suggestions for better practices. Optional fixes.

4. Protocol Summary

Mixoor is a Solana-based privacy mixer protocol built with the Anchor framework that enables private cryptocurrency transfers using zero-knowledge proofs. The protocol enables:

- **Private Deposits:** Users deposit SOL or SPL tokens and receive a cryptographic commitment
- **Anonymous Withdrawals:** Zero-knowledge proof verification for unlinkable withdrawals to any recipient
- **Merkle Tree Privacy:** Off-chain Merkle tree construction from on-chain events for scalability
- **Relayer Network:** Support for third-party relayers to enable withdrawals without on-chain linkability
- **Multi-Asset Support:** Separate pools for SOL and various SPL tokens

Key Components:

- Groth16 zero-knowledge proof system using Circom circuits
- Merkle tree with 20-level depth
- Nullifier mechanism to prevent double-spending
- Ring buffer of 50 historic Merkle roots for withdrawal validation
- Associated Token Account (ATA) integration for SPL tokens

5. Scope

The audit covered the complete Mixoor Protocol implementation. The repository was shared with Inversive Labs in a private capacity, and the entire codebase was in scope for security review.

Repository:

PROGRAM	COMMIT HASH
SmithiiDev/mixoor-program	`bdba885ff1c5d0c52adc5222ecee064d907e4c0d`

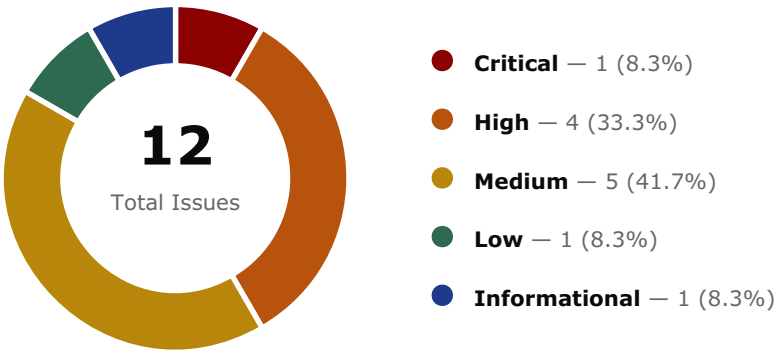
In Scope:

- **Solana Program** (`program/`):
  - All instruction handlers (5 modules: initialize, deposit, transfer, update_root, withdraw)
  - State account structures (Pool, NullifierAccount, Vault, RootEntry)
  - Utility functions (proof verification, merkle validation, cryptographic helpers)
  - Error definitions and constants
- **Circom Zero-Knowledge Circuits** (`circuits/`):
  - Transaction circuit with Merkle tree verification
  - Nullifier generation and commitment schemes
  - Groth16 proving system implementation

6. Executive Summary

The audit of the Mixoor Protocol identified **12 issues** across various severity levels. The most critical finding involves mint-agnostic deposits and withdrawals that enable direct fund theft. Additional high-severity findings include trusted setup weaknesses, root history manipulation, missing relayer fee transfers, and unrestricted admin withdrawals.

The following table summarizes the distribution of findings by severity:



7. Audit Revision Timeline

Initial Audit
Security assessment delivered

December 29, 2025

Round 1: Initial Fixes
Protocol team implements fixes

14 commits
e62f371 a91f250 68d2c54
73c2679 4ad45f2 8884c15
a25cbbf add2c08 bac471a
406f2ce 557921c 47ba533
4e55c7f 595ff6f 6dd471e

7 issues resolved
3 issues outstanding

Round 2: First Review
Auditor reviews fixes

Round 3: Second Fixes
Addressing 3 issues

3 commits
fbe778c 3f52e8b 090ef3b

1 issue resolved
2 issues remaining

Round 4: Second Review
Further refinement needed

Round 5: Final Fixes
Last 2 issues resolved

3 commits
e78c6e2 047ce25 892cfdd

8. Summary of Findings

#	ID	TITLE	SEVERITY
1	ILRV-2026-03.01	Mint-Agnostic Deposits and Withdrawals Enable Direct Theft of Pool Funds	CRITICAL
2	ILRV-2026-03.02	Trusted Setup Ceremony Contains Insufficient Participants	HIGH
3	ILRV-2026-03.03	Arbitrary Deposits Can Overwrite Merkle Roots and Freeze All Withdrawals	HIGH
4	ILRV-2026-03.04	Missing Relayer Fee Transfer for SPL Token Withdrawals	HIGH
5	ILRV-2026-03.05	Admin Withdraw Lacks Validation Enabling Fund Theft and Pool Uninitialization	HIGH
6	ILRV-2026-03.06	Unrestricted Pool Initialization Enables PDA Squatting and Future Token Griefing	MEDIUM
7	ILRV-2026-03.07	Missing Bit Length Constraints Enable Field Overflow in Amount Validation	MEDIUM
8	ILRV-2026-03.08	Admin Withdraw Does Not Update Pool Accounting	MEDIUM
9	ILRV-2026-03.09	Missing Merkle Tree Capacity Check Causes Permanent Fund Loss After 1M Deposits	MEDIUM
10	ILRV-2026-03.10	Missing Authorization Check Allows Malicious Relayers	MEDIUM
11	ILRV-2026-03.11	Root History Window Shrinks from 50 to 30 Entries	LOW
12	ILRV-2026-03.12	Security Comment May Benefit from Clarification	INFORMATIONAL

Critical

[ILRV-2026-03.01] Mint-Agnostic Deposits and Withdrawals Enable Direct Theft of Pool Funds

Summary

The `deposit`, `transfer`, and `withdraw` instructions accept any SPL mint provided by the caller without validating it matches `pool.mint`. Combined with the fact that commitments do not encode the mint (only `amount` and `poolId`), an attacker can deposit worthless tokens and immediately withdraw valuable tokens, directly stealing funds from honest users.

Description

During pool initialization, the selected mint is written to on-chain state, but none of the deposit or withdrawal instructions validate that the mint provided by the caller matches this stored value. This allows attackers to deposit worthless tokens into one token account and withdraw valuable tokens from a different token account owned by the same vault PDA.

Missing Validation in All Instructions:

The `deposit` instruction never validates the mint against `pool.mint`:

```
let depositor_token = ctx.accounts.depositor_token_account.as_ref()?;
let vault_token = ctx.accounts.vault_token_account.as_ref()?;
let mint = ctx.accounts.mint.clone();
// ...
token_interface::transfer_checked(cpi_context, amount, mint.decimals)?;
pool.total_deposited = pool.total_deposited.checked_add(amount)?;
emit!(CommitmentInserted { commitment, leaf_index, pool: ctx.accounts.pool.key() });
```

`deposit.rs#87-130`

The `transfer` instruction similarly lacks mint validation:

```
pub fn validate(&self, root: [u8; 32]) -> Result<()> {
  let pool = &self.pool.load()?;
  require!(pool.is_known_root(&root), MixoorErrorCode::UnknownRoot);
  // No check that self.mint.key() == pool.mint!
  // No check that vault_token_account.mint == pool.mint!
  Ok(())
}
```

`transfer.rs#75-86`

The same issue exists in the `withdraw` instruction, which only validates that the vault owns the token account but not that it's the correct mint.

Commitments Don't Encode Mint:

The zero-knowledge circuit computes commitments without including the mint:

```
commitment = Poseidon(secret, nullifier, amount, poolId)
```

`transaction.circom#76-81`

A commitment for "100 tokens" is indistinguishable regardless of whether those tokens are USDC or any other SPL token.

Attack Scenario:

1. Attacker mints worthless tokens (costs nothing)
2. Attacker deposits worthless tokens by calling `deposit(amount=1000, mint=X_MINT)`
    - Vault receives 1000 worthless X tokens in `ATA(vault, X)`
    - Valid commitment is emitted and inserted into Merkle tree
    - `pool.total_deposited += 1000`
3. Attacker immediately withdraws USDC using the same commitment:
    - Generates valid ZK proof for the commitment from step 2
    - Calls `transfer` with `mint=USDC_MINT` and `vault_token_account=ATA(vault, USDC)`
    - All validation passes (pool ID matches, root in history, proof verifies, vault owns both ATAs)
    - Attacker receives 1000 USDC while only depositing worthless tokens

Because both ATAs are owned by the same vault PDA, the `owner` constraint passes, but the protocol has no way to distinguish which ATA should be used for a given commitment.

Impact

- **Direct fund theft:** An attacker can deposit worthless tokens and immediately withdraw valuable tokens from the pool, stealing funds from honest users. The attack costs nearly nothing (just transaction fees) and can drain the entire pool.
- **Complete pool compromise:** Once the attack occurs, all legitimate user funds are stolen, pool state is corrupted, honest users' commitments become unspendable, and the Merkle tree is permanently poisoned with cross-mint commitments.
- **No capital requirement:** Unlike traditional MEV or economic attacks, this requires no upfront capital. The attacker only needs to mint worthless tokens and pay transaction fees (~0.001 SOL per deposit+withdrawal).
- **Instant and undetectable:** The attack completes in a single transaction bundle before any monitoring can react, and the stolen funds are immediately in the attacker's wallet.

Recommendation

Add mint validation constraints to all instructions:

For `deposit`:

```
+ use crate::state::Pool;

  #[derive(Accounts)]
  pub struct Deposit<'info> {
    #[account(mut)]
    pub pool: AccountLoader<'info, Pool>,

+   #[account(
+     constraint = mint.key() == pool.load()?.mint @ MixoorErrorCode::InvalidMint
+   )]
    pub mint: InterfaceAccount<'info, Mint>,

    #[account(
      mut,
      constraint = vault_token_account.owner == vault.key(),
+     constraint = vault_token_account.mint == pool.load()?.mint @ MixoorErrorCode::InvalidMint
    )]
    pub vault_token_account: Option<InterfaceAccount<'info, TokenAccount>>,
  }
```

For `transfer` **:**

```
  impl Transfer<'_> {
    pub fn validate(&self, root: [u8; 32]) -> Result<()> {
      let pool = &self.pool.load()?;
+
+     // Validate mint matches pool
+     require!(
+       self.mint.key() == pool.mint,
+       MixoorErrorCode::InvalidMint
+     );
+
+     // Validate vault token account mint
+     if let Some(vault_token) = &self.vault_token_account {
+       require!(
+         vault_token.mint == pool.mint,
+         MixoorErrorCode::InvalidMint
+       );
+     }

      require!(pool.is_known_root(&root), MixoorErrorCode::UnknownRoot);
      // ...
    }
  }
```

For `withdraw` **:**

```
  impl Withdraw<'_> {
    pub fn validate(&self) -> Result<()> {
+     let pool = self.pool.load()?;
+     require!(
+       self.mint.key() == pool.mint,
+       MixoorErrorCode::InvalidMint
+     );
      Ok(())
    }
  }
```

High

[ILRV-2026-03.02] Trusted Setup Ceremony Contains Insufficient Participants

Summary

The protocol uses a custom Powers of Tau ceremony with only 3 participants. Groth16's security model requires at least one participant to have honestly destroyed their toxic waste. A 3-person ceremony creates trust assumptions that may not be suitable for a production privacy protocol handling user funds.

Description

Groth16 proving systems require a two-phase trusted setup:

- **Phase 1 (Powers of Tau):** Universal parameters that benefit from many independent participants
- **Phase 2 (Circuit-specific):** Parameters tied to the specific circuit

The current ceremony was verified to contain only 3 contributions:

```
$ snarkjs powersoftau verify pot14_final.ptau
[INFO] Contribution #3: Coordinator
[INFO] Contribution #2: Participant 2
[INFO] Contribution #1: Participant 1
```

The file hash does not match any public ceremony, confirming this is a custom setup. With 3 participants who are likely colleagues, the security assumption becomes reliant on at least one team member having honestly destroyed their contribution randomness and never being compromised. Public ceremonies with hundreds of diverse, independent, and geographically distributed participants provide stronger security guarantees.

If toxic waste were recovered from all participants, it would be possible to forge arbitrary proofs, which could enable:

- Withdrawals without corresponding deposits
- Invalid Merkle membership proofs
- Nullifier forgery

Forged proofs would be cryptographically indistinguishable from legitimate ones.

Impact

If all participants' toxic waste were compromised, the soundness of the proving system would fail completely, potentially allowing unauthorized withdrawals from the pool. Exploitation would require insider compromise rather than an external attack vector. We recommend addressing this finding before mainnet deployment.

Recommendation

Replace the custom ceremony with the Hermez public Powers of Tau, which is widely used in production systems including Tornado Cash:

```
# Download Hermez ptau (2^14, 300+ participants)
wget https://hermez.s3-eu-west-1.amazonaws.com/powersOfTau28_hez_final_14.ptau

# Verify hash matches published value
sha256sum powersOfTau28_hez_final_14.ptau

# Regenerate proving key
snarkjs groth16 setup transaction.r1cs powersOfTau28_hez_final_14.ptau transaction_0000.zkey

# Optional: add team contribution to Phase 2
snarkjs zkey contribute transaction_0000.zkey transaction_final.zkey --name="Mixoor"

# Export and update verification key in program
snarkjs zkey export verificationkey transaction_final.zkey verification_key.json
```

Note that the verification key will change, requiring program redeployment. For additional assurance, the team may also consider conducting a public Phase 2 ceremony before mainnet launch, allowing community contributions to the circuit-specific setup.

[ILRV-2026-03.03] Arbitrary Deposits Can Overwrite Merkle Roots and Freeze All Withdrawals

Summary

The `deposit` instruction trusts the caller-provided `new_root`, appending it to `pool.root_history` without recomputing the Merkle tree or verifying that it matches the emitted `commitment`. A malicious user can repeatedly submit bogus roots and force every legitimate withdrawal to fail with `UnknownRoot` until operators manually repair the history.

Description

The protocol's Merkle tree architecture relies on user-provided roots for scalability, but this creates a DoS vulnerability:

User-Provided Root Accepted Without Validation:

```
pub fn deposit(ctx: Context<Deposit>, amount: u64, commitment: [u8; 32], new_root: [u8; 32]) -> Result<()> {
  Deposit::deposit(ctx, amount, commitment, new_root)
}
```

`lib.rs#34-41`

After transferring funds and emitting the commitment event, the program blindly accepts the root:

```
emit!(CommitmentInserted { commitment, leaf_index, pool: ctx.accounts.pool.key() });
pool.add_root(new_root, &Clock::get()?)?;
```

`deposit.rs#112-134`

The `add_root` function overwrites the current slot in a rolling buffer (modulo 30) without validation:

```
self.root_history[index] = RootEntry { root, timestamp: clock.unix_timestamp };
self.root_history_index = ((index + 1) % 30) as u8;
```

`state/mod.rs#36-45`

Attack Scenario:

An attacker can submit 30 deposits with arbitrary roots. Once the 30-slot buffer is filled with invalid roots, every honest withdrawal fails because the `transfer` instruction requires the proof's root to exist in the history:

```
require!(pool.is_known_root(&root), MixoorErrorCode::UnknownRoot);
```

`transfer.rs#75-85`

Impact

- **Complete withdrawal DoS:** Overwriting the 30-entry history prevents all users from withdrawing until valid roots push the bogus ones out or operators manually intervene.
- **Backend desynchronization:** The relayer rebuilds the tree off-chain and posts its own root via `update_root`, but malicious deposits can immediately overwrite it again, causing intermittent failures.
- **Phantom commitments:** Commitments emitted alongside fake roots remain unspendable (no honest root contains them), leaving balances that can never be withdrawn.

Recommendation

Validating the user-provided `new_root` on-chain is computationally infeasible due to Solana's compute limits. The following mitigations significantly reduce the DoS attack surface:

1. Increase Root History Buffer Size (Immediate Priority):

```
  pub struct Pool {
-   pub root_history: [RootEntry; 50],
+   pub root_history: [RootEntry; 500],
    // ...
  }

  pub fn add_root(&mut self, root: [u8; 32], clock: &Clock) -> Result<()> {
    let index = self.root_history_index as usize;
    self.root_history[index] = RootEntry { root, timestamp: clock.unix_timestamp };
-   self.root_history_index = ((index + 1) % 30) as u8;
+   self.root_history_index = ((index + 1) % 500) as u16;
    Ok(())
  }
```

2. Implement Minimum Deposit Amount (Short-term Priority):

```
+ const MIN_DEPOSIT_AMOUNT: u64 = 1_000_000; // e.g., 0.001 SOL or 0.001 USDC
+
  impl Deposit<'_> {
    pub fn validate(&self, amount: u64) -> Result<()> {
-     require!(amount > 0, MixoorErrorCode::InvalidAmount);
+     require!(amount >= MIN_DEPOSIT_AMOUNT, MixoorErrorCode::DepositTooSmall);
      Ok(())
    }
  }
```

3. Reserve Slots for Relayer-Provided Roots (Medium-term Priority):

Partition the root history into user-provided and relayer-provided sections to ensure the relayer can always maintain canonical roots even if attackers fill user slots.

[ILRV-2026-03.04] Missing Relayer Fee Transfer for SPL Token Withdrawals

Summary

The `transfer` instruction fails to pay the relayer fee for SPL token withdrawals, while correctly paying it for SOL withdrawals. This causes relayers to operate at a loss for SPL token transfers and corrupts pool accounting by overcounting `total_withdrawn`.

Description

The `transfer` instruction handles two asset types differently: native SOL and SPL tokens. While both paths validate the relayer fee in the zero-knowledge proof and update pool accounting identically, only the SOL path actually transfers the fee to the relayer.

SOL Withdrawal Path (Correct Implementation):

```
AssetType::Sol => {
  // To recipient (after protocol fee)
  ctx.accounts.vault.sub_lamports(final_recipient_amount)?;
  ctx.accounts.recipient.add_lamports(final_recipient_amount)?;

  // Protocol fee
  ctx.accounts.vault.sub_lamports(protocol_fee)?;
  ctx.accounts.fee_collector.add_lamports(protocol_fee)?;

  // Relayer fee
  if relayer_fee > 0 {
    ctx.accounts.vault.sub_lamports(relayer_fee)?;  // ✓ Paid
    ctx.accounts.relayer.add_lamports(relayer_fee)?; // ✓ Paid
  }
}
```

`transfer.rs#142-159`

Three transfers occur: recipient receives funds, fee collector receives protocol fee, and relayer receives their fee.

SPL Token Withdrawal Path (Missing Relayer Fee):

```
AssetType::SplToken => {
  // Transfer to recipient (after protocol fee)
  token_interface::transfer_checked(cpi_context, final_recipient_amount, mint.decimals)?;

  // Transfer to fee collector
  token_interface::transfer_checked(cpi_context, protocol_fee, mint.decimals)?;

  // ✗ Relayer fee transfer is missing!
}
```

`transfer.rs#160-241`

Only two transfers occur: recipient and fee collector. The relayer receives nothing despite paying transaction fees and ATA creation costs.

Accounting Impact:

Both paths incorrectly update `total_withdrawn` by including the unpaid relayer fee:

```
let total_withdrawn_amount = recipient_amount
  .checked_add(relayer_fee)  // Overcounts for SPL tokens
  .ok_or(MixoorErrorCode::Overflow)?;

pool.total_withdrawn = pool.total_withdrawn
  .checked_add(total_withdrawn_amount)
  .ok_or(MixoorErrorCode::Overflow)?;
```

`transfer.rs#244-252`

For SPL tokens, the claimed withdrawn amount (`recipient_amount + relayer_fee`) exceeds the actual amount withdrawn (`recipient_amount`), leaving the `relayer_fee` in the vault but counting it as withdrawn.

Impact

- **Relayers operate at a loss:** Relayers pay transaction fees and ATA creation costs (~0.002 SOL + gas) but receive zero compensation for SPL token transfers.
- **Economic attack:** No rational actor will relay SPL withdrawals, preventing users from withdrawing SPL tokens without self-relaying (which defeats the privacy purpose).
- **Corrupted accounting:** `total_withdrawn` permanently diverges from actual withdrawals, breaking solvency monitoring and pool analytics.

Recommendation

Add the missing relayer fee transfer in the SPL token case:

```
  token_interface::transfer_checked(cpi_context, protocol_fee, mint.decimals)?;
+
+   // Transfer relayer fee
+   if relayer_fee > 0 {
+     let relayer_token_account = ctx
+       .accounts
+       .relayer_token_account  // ← Need to add this account to Transfer struct
+       .as_ref()
+       .ok_or(MixoorErrorCode::MissingTokenAccount)?;
+
+     let relayer_cpi_accounts = TransferChecked {
+       from: vault_token_account.to_account_info().clone(),
+       mint: mint.to_account_info().clone(),
+       to: relayer_token_account.to_account_info(),
+       authority: ctx.accounts.vault.to_account_info(),
+     };
+     let cpi_context = CpiContext::new_with_signer(
+       cpi_program.to_account_info(),
+       relayer_cpi_accounts,
+       signer_seeds,
+     );
+     token_interface::transfer_checked(cpi_context, relayer_fee, mint.decimals)?;
+   }
  }
```

[ILRV-2026-03.05] Admin Withdraw Lacks Validation Enabling Fund Theft and Pool Uninitialization

Summary

The `withdraw` instruction allows the admin to extract arbitrary amounts from the vault without any validation checks. This enables two critical attack vectors: (1) admin can steal user deposits while users have unspent commitments, causing permanent fund loss, and (2) admin can drain the SOL vault to exactly 0 lamports, causing the account to be uninitialized and reclaimed by the System program.

Description

The `withdraw` instruction has an empty validation function and performs no checks on the withdrawal amount:

```
impl Withdraw<'_> {
  pub fn validate(&self) -> Result<()> {
    Ok(())  // Empty validation!
  }

  #[access_control(ctx.accounts.validate())]
  pub fn withdraw(ctx: Context<Withdraw>, amount: u64) -> Result<()> {
    let pool = ctx.accounts.pool.load()?;
    // ... no validation on amount parameter ...

    match asset_type {
      AssetType::Sol => {
        ctx.accounts.vault.sub_lamports(amount)?;  // No checks!
        ctx.accounts.authority.add_lamports(amount)?;
      }
      AssetType::SplToken => {
        // ... SPL token transfer with no checks ...
      }
    }

    Ok(())  // No accounting update
  }
}
```

`withdraw.rs#62-129`

Missing Validations:

- No check that `amount > 0`
- No check that `amount <= excess_funds` (funds not belonging to users)
- No check that withdrawal maintains vault rent-exemption (SOL pools)
- No check against solvency
- Empty `validate()` function

Attack Scenario 1: Steal User Deposits

1. User deposits 1,000 USDC, creates commitment with unspent nullifier
2. Admin calls `withdraw(1_000_000_000)`, transferring 1,000 USDC to admin wallet
3. Vault now has 0 USDC, but `pool.total_withdrawn` is still 0
4. User attempts withdrawal with valid ZK proof → fails with `InsufficientFunds`
5. Result: User's 1,000 USDC permanently lost

Attack Scenario 2: Drain Vault to Zero (SOL pools)

1. Vault has exactly rent-exemption minimum (1,837,440 lamports)
2. Admin calls `withdraw(1_837_440)`
3. Solana runtime allows transition to 0 lamports → account closes
4. All future operations fail with "Account not found"
5. Pool permanently bricked, cannot be reinitialized due to PDA collision

Impact

- **Direct fund theft:** Admin can withdraw user deposits while users hold unspent commitments, causing permanent loss of user funds with no on-chain recourse.

- **Permanent pool destruction:** Admin can drain SOL vault to exactly 0 lamports, causing the account to close and permanently bricking all pool operations.
- **No validation guardrails:** The empty `validate()` function creates a dangerous footgun even for honest admins who might accidentally brick the pool through miscalculation.

Recommendation

Option 1: Remove the instruction entirely (Recommended)

For a trustless privacy mixer, there should be no admin escape hatch:

```
- pub fn withdraw(ctx: Context<Withdraw>, amount: u64) -> Result<()> {
-   // ... entire implementation ...
- }
```

Option 2: Restrict to excess funds only

If emergency withdrawal is required, enforce strict validation:

```
pub fn validate(&self) -> Result<()> {
    let pool = self.pool.load()?;

    // Calculate funds belonging to users
    let user_funds = pool.total_deposited
        .checked_sub(pool.total_withdrawn)
        .ok_or(MixoorErrorCode::Underflow)?;

    // Get vault balance
    let vault_balance = get_vault_balance(self)?;

    // Calculate excess (protocol fees, donations, etc.)
    let excess = vault_balance
        .checked_sub(user_funds)
        .ok_or(MixoorErrorCode::InsufficientExcess)?;

    require!(excess > 0, MixoorErrorCode::NoExcessFunds);
    Ok(())
}

pub fn withdraw(ctx: Context<Withdraw>, amount: u64) -> Result<()> {
    let mut pool = ctx.accounts.pool.load_mut()?;

    // Enforce amount <= excess funds
    let user_funds = pool.total_deposited.checked_sub(pool.total_withdrawn)?;
    let vault_balance = get_vault_balance(&ctx)?;
    let excess = vault_balance.checked_sub(user_funds)?;

    require!(amount <= excess, MixoorErrorCode::CannotWithdrawUserFunds);

    // For SOL pools, enforce rent-exemption
    if asset_type == AssetType::Sol {
        let rent = Rent::get()?;
        let min_rent = rent.minimum_balance(Vault::LEN);
        require!(
            vault_balance.checked_sub(amount)? >= min_rent,
            MixoorErrorCode::WouldLoseRentExemption
        );
    }

    // Proceed with withdrawal and update accounting
    pool.total_withdrawn = pool.total_withdrawn.checked_add(amount)?;
    Ok(())
}
```

Medium

[ILRV-2026-03.06] Unrestricted Pool Initialization Enables PDA Squatting and Future Token Griefing

Summary

The `initialize` instruction does not restrict who can create pools, allowing any user to initialize pools for arbitrary token mints. While this does not affect pools already deployed by the Mixoor team (SOL and USDC), it enables attackers to preemptively occupy PDAs for future tokens, preventing the protocol from officially supporting new assets without redeploying the entire program.

Description

The `initialize` instruction uses a permissionless design where any signer can create a pool for any token mint. The pool PDA is derived using `seeds = [b"pool", mint.key(), asset_type]`, creating a deterministic address that can only be initialized once.

```
// ? do we need to add an address constraint
#[account(mut)]
pub authority: Signer<'info>,

#[account(
  init,
  payer = authority,
  space = Pool::LEN,
  seeds = [b"pool",mint.key().as_ref(),asset_type.as_u8().to_le_bytes().as_ref()],
  bump
)]
pub pool: AccountLoader<'info, Pool>,
```

`initialize.rs#18-32`

The function stores the caller as the pool authority without validation:

```
pub fn initialize(ctx: Context<Initialize>, asset_type: AssetType) -> Result<()> {
  let mut pool = ctx.accounts.pool.load_init()?;
  pool.authority = ctx.accounts.authority.key();  // Stores arbitrary caller
  // ...
}
```

Root Cause: The PDA derivation excludes the authority from the seeds, meaning only one pool can exist per `(mint, asset_type)` pair regardless of who creates it. This creates a race condition where the first caller to initialize a pool for a given mint becomes its permanent authority.

Attack Vector:

1. Mixoor team announces support for USDT
2. Attacker front-runs deployment by calling `initialize(authority: ATTACKER, mint: USDT_MINT, asset_type: SplToken)`
3. Pool PDA is created with `pool.authority = ATTACKER`
4. Team attempts official deployment → Error: Account already initialized

5. Team cannot create official USDT pool without redeploying the entire program with a new program ID

Impact

For already deployed pools (SOL, USDC): No impact - pools are already controlled by the Mixoor team.

For future token expansion:

- **PDA squatting:** Attackers can occupy PDAs for popular tokens, blocking official deployment
- **Protocol expansion blocked:** Team cannot add new token support without program redeployment
- **Operational overhead:** Requires redeploying program, updating all frontends, and migrating users
- **Fragmented liquidity:** Old and new program instances would coexist, splitting user base
- **Phishing risk:** Attackers can create legitimate-looking pools and build fake frontends

Recommendation

Add an address constraint to restrict pool creation to a trusted authority:

```
+use anchor_lang::solana_program::pubkey;
+
+// Define trusted admin address
+const ADMIN_ADDRESS: Pubkey = pubkey!("YourMixoorTeamPublicKeyHere");
+
 #[derive(Accounts)]
 #[instruction(asset_type: AssetType)]
 pub struct Initialize<'info> {
-  // ? do we need to add an address constraint
-  #[account(mut)]
+  #[account(
+    mut,
+    address = ADMIN_ADDRESS @ MixoorErrorCode::Unauthorized
+  )]
   pub authority: Signer<'info>,

   #[account(
     init,
     payer = authority,
     space = Pool::LEN,
     seeds = [b"pool",mint.key().as_ref(),asset_type.as_u8().to_le_bytes().as_ref()],
     bump
   )]
   pub pool: AccountLoader<'info, Pool>,
   // ...
 }
```

This fix ensures: - Only the Mixoor team can create official pools - Future token expansion is protected from front-running - No program redeployment required to add new tokens - Consistent with security best practices for mixer protocols (Tornado Cash model)

[ILRV-2026-03.07] Missing Bit Length Constraints Enable Field Overflow in Amount Validation

Summary

The circuit's amount validation logic (`GreaterEqThan(64)`) does not constrain its inputs to 64-bit values, allowing field arithmetic overflow. While the Solana program mitigates this by restricting inputs to u64 values, the circuit itself is vulnerable. This is rated MEDIUM because exploitation is prevented in the current deployment, but the architectural pattern is not advised for ZK circuits.

Description

The circuit validates that deposited funds cover withdrawal fees and recipient amount, but fails to constrain bit lengths before arithmetic:

```
// Verify amount covers fee + refund
component amountCheck = GreaterEqThan(64);
amountCheck.in[0] <== amount;
amountCheck.in[1] <== fee + refund;  // No range check before addition!
amountCheck.out === 1;
```

`transaction.circom#99-102`

The circuit accepts `fee` and `refund` as arbitrary field elements (up to ~2^254), which can cause wraparound in field arithmetic. It was validated that setting `fee = p - 100` and `refund = 200` results in `fee + refund ≡ 100 (mod p)`, bypassing the amount check.

However, the Solana program validates these values against u64 instruction parameters:

```
// Validate relayer fee (index 4)
let expected_fee = u64_to_field_bytes(relayer_fee);  // Converts u64
require!(public_inputs[4] == expected_fee, MixoorErrorCode::PublicInputMismatch);

// Validate recipient amount (index 5)
let expected_amount = u64_to_field_bytes(recipient_amount);  // Converts u64
require!(public_inputs[5] == expected_amount, MixoorErrorCode::PublicInputMismatch);
```

`utils/helpers.rs#75-86`

With u64 values, `fee + refund` cannot overflow the BN254 field order (2^254).

Impact

- **Circuit level:** Vulnerable to field overflow bypass
- **System level:** Mitigated by Solana validation - no immediate security risk
- **Code quality:** ZK circuits should be self-contained and trustless, not rely on external validation
- **Portability risk:** If the circuit is forked or the Solana validation is removed, the vulnerability becomes exploitable

Recommendation

Add explicit bit length constraints in the circuit:

```
 // Verify amount covers fee + refund
+// Constrain inputs to 64 bits to prevent field overflow
+component amountBits = Num2Bits(64);
+amountBits.in <== amount;
+
+component feeBits = Num2Bits(64);
+feeBits.in <== fee;
+
+component refundBits = Num2Bits(64);
+refundBits.in <== refund;
+
+// Now safe to use GreaterEqThan
 component amountCheck = GreaterEqThan(64);
 amountCheck.in[0] <== amount;
 amountCheck.in[1] <== fee + refund;
 amountCheck.out === 1;
```

[ILRV-2026-03.08] Admin Withdraw Does Not Update Pool Accounting

Summary

The `withdraw` instruction transfers funds from the vault but does not update `pool.total_withdrawn`, breaking the protocol's accounting invariants. This makes vault insolvency undetectable by off-chain monitoring systems.

Description

The withdraw function transfers funds but returns without updating accounting:

```
pub fn withdraw(ctx: Context<Withdraw>, amount: u64) -> Result<()) {
    let pool = ctx.accounts.pool.load()?;  // Immutable load
    // ... transfer funds ...
    Ok(())  // No accounting update
}
```

`withdraw.rs#68-128`

The protocol maintains the invariant `vault_balance = total_deposited - total_withdrawn`. After admin withdrawal, this breaks:

```
Before: vault_balance = 1,000,000, total_deposited = 1,000,000, total_withdrawn = 0 ✓
Admin withdraws 100,000
After: vault_balance = 900,000, total_deposited = 1,000,000, total_withdrawn = 0 ✗
```

Monitoring systems relying on `total_withdrawn` cannot detect insolvency. There is no on-chain record of admin withdrawals.

Impact

- **Broken accounting invariants:** Impossible to verify pool health through on-chain state alone
- **Undetectable insolvency:** Monitoring fails to detect insufficient funds
- **Corrupted analytics:** TVL, fee tracking, and metrics become inaccurate
- **Audit trail loss:** No on-chain record of admin withdrawals

Recommendation

Update accounting when admin withdraws:

```
  pub fn withdraw(ctx: Context<Withdraw>, amount: u64) -> Result<()> {
-   let pool = ctx.accounts.pool.load()?;
+   let mut pool = ctx.accounts.pool.load_mut()?;

    // transfer logic ...

+   pool.total_withdrawn = pool
+       .total_withdrawn
+       .checked_add(amount)
+       .ok_or(MixoorErrorCode::Overflow)?;

    Ok(())
  }
```

Additionally, emit an event for auditability:

```
#[event]
pub struct AdminWithdrawal {
    pub pool: Pubkey,
    pub admin: Pubkey,
    pub amount: u64,
    pub timestamp: i64,
}
```

[ILRV-2026-03.09] Missing Merkle Tree Capacity Check Causes Permanent Fund Loss After 1M Deposits

Summary

The `deposit` instruction does not validate that `next_leaf_index < 2^20`. The ZK circuit supports a maximum of 1,048,576 leaves. Once this capacity is reached, subsequent deposits succeed on-chain but can never be withdrawn, resulting in permanent, irreversible loss of user funds.

Description

The ZK circuit is compiled with a fixed tree depth of 20:

```
component main {public [root, nullifierHash, recipient, relayer, fee, refund, poolId]} = Transaction(20);
```

`transaction.circom#110`

This supports exactly 2^20 = 1,048,576 leaf positions (indices 0-1,048,575). However, the deposit instruction doesn't enforce this limit:

```
let leaf_index = pool.next_leaf_index;  // Could be >= 1,048,576

emit!(CommitmentInserted {
  commitment,
  leaf_index,  // No validation
  pool: ctx.accounts.pool.key(),
});

pool.next_leaf_index = pool.next_leaf_index.checked_add(1)?;  // Only checks u32 overflow
```

`deposit.rs#112-125`

After deposit #1,048,577, users cannot generate valid proofs because: - A depth-20 tree only has positions 0-1,048,575 - The circuit expects exactly 20 path elements - Proofs with 21 elements are rejected

The error `MixoorErrorCode::MerkleTreeFull` exists but is never used.

Impact

- **Permanent fund loss:** After 1,048,576 deposits, all subsequent deposits are irreversibly lost
- **Delayed timeline:** Could take months to years depending on volume
- **Invisible to users:** On-chain transaction succeeds, no indication funds are unrecoverable
- **No recovery mechanism:** Only solution is to deploy new program with deeper circuit

Recommendation

Add capacity check before accepting deposits:

```
  pub fn deposit(...) -> Result<()> {
    let mut pool = ctx.accounts.pool.load_mut()?;
    let leaf_index = pool.next_leaf_index;

+   // Enforce tree capacity (depth 20 = 2^20 leaves)
+   require!(
+     leaf_index < 1_048_576,
+     MixoorErrorCode::MerkleTreeFull
+   );

    emit!(CommitmentInserted { commitment, leaf_index, pool: ctx.accounts.pool.key() });
    pool.next_leaf_index = pool.next_leaf_index.checked_add(1)?;
  }
```

Alternative - define constant:

```
// In constants.rs
pub const MAX_TREE_DEPTH: u32 = 20;
pub const MAX_TREE_LEAVES: u32 = 1 << MAX_TREE_DEPTH; // 1,048,576

// In deposit.rs
require!(leaf_index < MAX_TREE_LEAVES, MixoorErrorCode::MerkleTreeFull);
```

[ILRV-2026-03.10] Missing Authorization Check Allows Malicious Relayers

Summary

The `transfer` instruction is missing the `RELAYER_ADDRESS` constraint that exists in `update_root`, allowing any address to sign withdrawal transactions. This creates a phishing/social engineering vector where users who trust malicious third-party relayer services could have their funds stolen.

Description

The protocol defines a trusted relayer address used to restrict `update_root`:

```
#[constant]
pub const RELAYER_ADDRESS: Pubkey = pubkey!("re1R63DiMLtDyHMAS1ohszqfFTHcF9Q3uEXSYdHzHWU");
```

`utils/constants.rs#28-34`

However, the `transfer` instruction is missing this constraint:

```
#[derive(Accounts)]
pub struct Transfer<'info> {
  #[account(mut)]  // anyone can be the relayer
  pub relayer: Signer<'info>,
  // ...
}
```

`transfer.rs#18-19`

The ZK proof generation does not require special credentials - circuit artifacts (.wasm, .zkey) are public and anyone can generate proofs using `snarkjs`. The "relayer" is simply the account that signs and pays for the transaction. Without the address constraint, anyone can act as a relayer.

Impact

- **Direct theft:** Users who withdraw through malicious relayers lose funds permanently
- **Attack difficulty:** Trivial - attacker sets up a relayer service
- **User awareness:** Zero - users cannot detect attack until funds are gone
- **Reversibility:** None - nullifier is spent, user cannot retry

Rated MEDIUM because it requires user to choose non-official relayer service (phishing/social engineering), making likelihood low.

Recommendation

Restrict relayer to authorized address:

```
  #[derive(Accounts)]
  pub struct Transfer<'info> {
-   #[account(mut)]
+   #[account(
+     mut,
+     address = RELAYER_ADDRESS @ MixoorErrorCode::Unauthorized
+   )]
    pub relayer: Signer<'info>,
```

This forces all withdrawals through the trusted relayer service.

Low

[ILRV-2026-03.11] Root History Window Shrinks from 50 to 30 Entries

Summary

`Pool::root_history` reserves 50 slots, but `add_root` increments the ring-buffer index modulo 30. As soon as 30 deposits/withdrawals occur, the oldest root is evicted even though 20 slots remain unused. Users and relayers that rely on the documented 50-root history will see valid proofs rejected on-chain because their roots disappear earlier than expected.

Description

The on-chain pool state allocates space for 50 historic Merkle roots:

```
pub struct Pool {
  pub root_history: [RootEntry; 50],
  // ...
  pub root_history_index: u8,
}
```

`state/mod.rs#6-34`

However, `Pool::add_root` manually wraps `root_history_index` with `% 30`, so the ring buffer cycles through only 30 entries:

```
pub fn add_root(&mut self, root: [u8; 32], clock: &Clock) -> Result<()> {
  let index = self.root_history_index as usize;
  self.root_history[index] = RootEntry { root, timestamp: clock.unix_timestamp };
  self.root_history_index = ((index + 1) % 30) as u8;
  Ok(())
}
```

`state/mod.rs#36-43`

The `initialize` instruction clears all 50 entries on pool creation, reinforcing the expectation that 50 roots should stay available:

```
pool.root_history = [RootEntry::default(); 50];
pool.root_history_index = 0;
```

`initialize.rs#88-94`

Root cause: The modulo constant (30) does not match the allocated array length (50). Twenty slots are never referenced, so the oldest root expires after 30 updates instead of 50.

Impact

- **Reduced availability:** A sustained flow of deposits/withdrawals rotates the buffer after only 30 events, so proofs generated hours earlier may fail with `UnknownRoot`
- **Smaller anonymity window:** Users expect ~50 recent roots to remain valid; by expiring them after 30, the effective participant set shrinks
- **Operational friction:** Relayers and clients must refresh proofs more often than planned

Severity **Low**: funds are not at risk, but protocol reliability degrades under normal or bursty usage.

Recommendation

Align the modulo with the actual buffer size:

```
  pub fn add_root(&mut self, root: [u8; 32], clock: &Clock) -> Result<()> {
    let index = self.root_history_index as usize;
    self.root_history[index] = RootEntry { root, timestamp: clock.unix_timestamp };
-   self.root_history_index = ((index + 1) % 30) as u8;
+   self.root_history_index = ((index + 1) % 50) as u8;
    Ok(())
  }
```

Alternatively, reduce the array to 30 entries if that is the intended window, and document the limit so clients know how long roots remain valid.

Informational

[ILRV-2026-03.12] Security Comment May Benefit from Clarification

Summary

The `withdraw` instruction contains a comment describing a potential attack scenario that appears to be prevented by the protocol's nullifier mechanism. Clarifying this comment could help avoid confusion about the instruction's actual security considerations.

Description

The comment in the withdraw instruction describes a concern about user double-spending:

```
/*
 * NOTE: This should be used sparingly and ideally not exist here.
 *
 * Attack vector: A user might call deposit and retain their nullifier,
 * secret, and commitment. After receiving funds from the admin calling
 * this instruction—and with the root from their deposit still in the
 * ring buffer—they could call transfer, resulting in double-spending.
 */
```

`withdraw.rs#14-21`

However, the protocol's nullifier mechanism prevents the described scenario. When a user withdraws, a nullifier PDA is created with the `init` constraint:

```
#[account(
    init,  // Fails if nullifier already exists
    payer = relayer,
    space = 8 + 32 + 8,
    seeds = [b"nullifier", pool.key().as_ref(), nullifier_hash.as_ref()],
    bump
)]
pub nullifier: Account<'info, NullifierAccount>,
```

`transfer.rs#36-42`

Since the PDA uses `init`, attempting to use the same nullifier twice would fail with "Account already exists", preventing double-spending regardless of admin actions.

The more relevant security consideration for this instruction is that it allows withdrawing funds from the vault without validation that those funds exceed user deposits (addressed in H-04).

Impact

This is an informational issue about code documentation:

- The comment may cause confusion about which security properties the protocol relies on
- Future auditors or developers might spend time analyzing the mentioned scenario

- The actual security considerations of the instruction (covered in H-04) are not mentioned in the comment

Recommendation

Update the comment to clarify the actual security considerations:

```
/*
 * WARNING: Admin emergency withdrawal function.
 *
 * Security consideration: This instruction can withdraw funds from the vault
 * without verifying that the amount doesn't exceed user deposits. See the
 * validate() function and consider adding checks to prevent withdrawing
 * funds that belong to users with unspent commitments.
 *
 * Note: User double-spending is prevented by the nullifier mechanism in the
 * transfer instruction, which uses the 'init' constraint to ensure each
 * nullifier can only be used once.
 */
```

This change would help clarify the instruction’s security model without changing its functionality.